

Validación y Verificación (V & V)

V & V son dos conceptos clave que se aplican en distintas etapas del ciclo de vida del software, asegurando que el producto cumple tanto con las especificaciones como con las necesidades del usuario.

- **Verificación:**
 - **Propósito:** Confirmar si el software **cumple las especificaciones de diseño y requisitos definidos** ("¿Estamos construyendo el producto correctamente?").
 - **Enfoque:** Se basa en los **requisitos iniciales de análisis y diseño**.
 - **Cuándo:** Principalmente en las **primeras etapas** del ciclo de vida.
- **Validación:**
 - **Propósito:** Probar que el software **satisface las necesidades reales de los usuarios finales** ("¿Estamos construyendo el producto correcto?").
 - **Enfoque:** Asegura que el producto esté **listo para su uso** y cumpla las expectativas comerciales.
 - **Cuándo:** Se lleva a cabo en las **fases finales** del desarrollo, usando pruebas enfocadas en la satisfacción del usuario (ej: UAT).

Definiciones de Testing

- "Es la **verificación dinámica** de la adecuación del sistema a los **requerimientos**."
- "El infinito proceso de **comparar lo invisible con lo ambiguo**."
- "Una forma efectiva de **mostrar la presencia de errores, pero inadecuada para mostrar su ausencia**."

Riesgo

Un **evento futuro incierto** que, si ocurre, tiene una **incidencia** (impacto). Sus características son:

- **Probabilidad de ocurrencia.**
- **Impacto.**

Tipos de Riesgo:

- **Negocio:** Afecta a la organización.
- **Proyecto:** Afecta plazos, costos o alcance.
- **Producto:** Afecta calidad o desempeño del producto.
- **Externos:** Fuera del control directo.

Calidad

- **RAE:** Propiedades de algo que permiten apreciarlo como mejor o peor.
- **ISO:** Grado en que las características de algo cumplen con los requisitos.

Gestión de Calidad (QM - Quality Management):

- **QP (Quality Planning):** Define requisitos y estrategias de calidad.
- **QA (Quality Assurance):** Evalúa y documenta procesos para **prevenir problemas** y asegurar la calidad.
- **QC (Quality Control):** **Inspecciona productos/servicios** para identificar y corregir defectos.
- **QI (Quality Improvement):** Detección y análisis para la **mejora continua**.

Características de los Modelos de Calidad

1. Son de carácter **obligatorio** (si se adoptan).
2. Funcionan como **marco de referencia**.
3. **Adaptables** a cualquier tipo de empresa.
4. Son **autoevaluables**.
5. **No son certificables** (a diferencia de las normas).

Diferencia entre Depuración (Debugging) y Pruebas (Testing)

- **Pruebas (Testing):** Actividades para **identificar defectos**.
 - **Dinámicas:** Ejecutan el software, **descubren fallos** que revelan defectos.
 - **Estáticas:** Analizan artefactos (ej. código) **directamente para encontrar defectos**.
- **Depuración (Debugging):** Proceso de **identificar, analizar y eliminar la causa raíz** de un fallo o defecto encontrado.
 - Si un **test dinámico** provoca un fallo, se inicia la depuración para encontrar el error subyacente.
 - Con **tests estáticos**, el hallazgo del defecto a menudo facilita su eliminación inmediata.

Tester Agile

Un **Tester Agile** actúa como un **enlace clave (comunicador)** entre el cliente (negocio) y los desarrolladores. Entiende los requisitos del cliente y la complejidad técnica para asegurar que se construya la solución correcta. Trabaja **colaborativamente** con los

desarrolladores, con comunicación frecuente y entregando software funcional regularmente. Sí, a menudo se asocia con el rol de **QA** en equipos ágiles.

Atributos de Calidad

- **Funcionales:**
 - **Correctitud:** El sistema hace lo que debe hacer.
 - **Compleitud:** El sistema implementa todos los requisitos.
 - **No Funcionales:**
 - Fiabilidad, Usabilidad, Portabilidad, Eficiencia, Mantenibilidad, Seguridad.
-

¿Cuánto Testing es Necesario?

Dado que **probar todas las combinaciones es imposible**, la cantidad de testing se define priorizando basándose en:

1. **Riesgos:** Qué áreas son críticas o tienen mayor probabilidad de fallar.
 2. **Plazos y Presupuesto:** Las limitaciones de tiempo y recursos disponibles.
-

7 principios del Testing

1. **Las pruebas muestran defectos, pero no su ausencia.**
 2. **Las pruebas exhaustivas son imposibles:** Siempre hay que priorizar qué probar.
 3. **Probar temprano ahorra dinero:** Detectar defectos al inicio del ciclo de vida es más barato.
 4. **Los defectos se agrupan:** Pocos módulos suelen contener la mayoría de los defectos.
 5. **Paradoja del pesticida:** Si los mismos tests se repiten, dejan de encontrar nuevos defectos; hay que actualizarlos.
 6. **El testing depende del contexto:** El enfoque de prueba varía según el tipo de software.
 7. **Falacia de la ausencia de errores:** Un software sin defectos obvios no sirve si no satisface las necesidades del usuario.
-

Niveles de Dependencia de las Pruebas

Estos se refieren a quién diseña o de dónde provienen las pruebas:

1. Diseñadas por el **desarrollador** (ej. unitarias).
2. Diseñadas por **terceros** (ej. librerías externas).

3. Diseñadas por **alguien de otra área interna** (ej. QA, analistas de negocio).
4. Diseñadas por **alguien de otra organización** (ej. auditorías externas, pruebas de integración con socios).

Etapas del Desarrollo de Software

El ciclo de vida del software tradicionalmente se divide en: **Análisis -> Diseño -> Desarrollo -> Testing -> Implementación -> Mantenimiento.**

Roles en el Desarrollo

Los roles clave en el proceso son: **Analista Funcional/Product Owner -> Diseñador UX/UI -> Programador -> Testers/QA -> DevOps/Usuarios.** La **planificación permanente** es liderada por el **Scrum Master / Líder de Proyecto.**

Modelos de Ciclo de Vida del Software

Estos procesos se implementan a través de distintos modelos:

- **Cascada (Tradicional):** Cada etapa se completa antes de pasar a la siguiente, de forma secuencial.
- **Espiral:** Un enfoque iterativo y basado en el riesgo, con cuatro fases (planificación, desarrollo, análisis de riesgo, determinación de objetivos).
- **Prototipos:** Se basa en la creación rápida y el refinamiento de prototipos interactivos con el usuario.
- **Iterativos:** El software se construye en ciclos repetitivos. Cada iteración incluye análisis, diseño, desarrollo y pruebas, refinando el producto.
- **Incrementales:** Similar al iterativo, pero cada iteración entrega una parte **funcional y creciente** del producto final.

Historias de Usuario (HU)

Son una **descripción informal de una función de software desde la perspectiva del usuario.** Las redacta el **Product Owner (PO)** y sirven para que los equipos entiendan el trabajo a realizar.

- **Formato:**
 - Como [perfil] quiero [objetivo del software] para lograr [resultado].
-

Criterios de Aceptación

Definen **cuándo una Historia de Usuario está terminada**. Se escriben como escenarios detallados:

- **Escenario:**
 - **Como** [perfil] **quiero** [objetivo del software] **para lograr** [resultado].
 - **Dado que:** [Condiciones previas]
 - **Cuando:** [El momento o acción precisa]
 - **Entonces:** [Lo que debe suceder con detalle]
-

HU(Historias de usuario)Efectivas: 3C

Para que una HU sea efectiva, se basa en los 3 pilares:

- **Card:** Un formato físico (ej. *post-it*) que representa la historia.
 - **Conversation:** Discusiones continuas con el equipo para aclarar la historia.
 - **Confirmation:** Los criterios de aceptación que confirman que la solución es correcta.
-

Principios INVEST de las HU

Una buena Historia de Usuario debe ser:

- **Independiente:** No depende de otras HU.
- **Negociable:** Permite discutir los detalles de implementación.
- **Valiosa:** Aporta valor al usuario o negocio.
- **Estimable:** Se puede estimar su tamaño o esfuerzo.
- **Small (Pequeña):** Lo suficientemente pequeña para ser completada en una iteración.
- **Testable:** Se pueden definir criterios de aceptación claros para probarla.

Metodologías Ágiles

Características clave (Manifiesto Ágil):

- **Individuos e interacciones** sobre procesos y herramientas.
- **Software funcionando** sobre documentación extensiva.
- **Colaboración con el cliente** sobre negociación contractual.
- **Respuesta al cambio** sobre seguir un plan.

12 principios (Resumidos):

1. Prioridad: **satisfacer al cliente** con entregas tempranas y continuas de software.

2. **Aceptación de cambios**, incluso tardíos.
3. Entregar software funcional en **períodos cortos**.
4. Colaboración diaria entre **negocio y desarrolladores**.
5. **Motivación y soporte** a los individuos. (Tu "laburantes felices y limpios" está bien en su espíritu).
6. La **conversación cara a cara** es el método más eficiente de comunicación.
7. El **software funcionando** es la principal medida de progreso.
8. Promover el **desarrollo sostenible y un ritmo constante**.
9. Búsqueda continua de **excelencia técnica y buen diseño**.
10. La **simplicidad** es esencial (maximizar el trabajo no hecho).
11. Las mejores arquitecturas y diseños emergen de **equipos autoorganizados**.
12. El equipo se **auto-reflexiona** para mejorar su efectividad.

Manifiesto Ágil para Testing (Enfoque QA):

- **Testing durante** el desarrollo > Testing al final.
 - **Prevenir bugs** > Encontrar bugs.
 - **Entender qué se testea** (validación) > Verificar funcionalidad (verificación).
 - **Construir mejor sistema** > Romper el sistema.
 - **El equipo es responsable de la calidad** > El tester es el único responsable.
-

Modelo V y Modelo W

- **Modelo V:**
 - Similar al **modelo en cascada**.
 - Las fases de **prueba corresponden directamente con las fases de desarrollo**.
 - Cada fase se completa antes de iniciar la siguiente, de forma secuencial y en forma de "V".
- **Modelo W:**
 - Las **actividades de prueba se desarrollan en paralelo** al proceso de desarrollo.
 - Enfatiza la **integración temprana** y continua de las pruebas en cada etapa.
 - Generalmente se representa con **dos "V" entrelazadas**, indicando que por cada fase de desarrollo hay una fase de prueba paralela.

Descomposición de Historias de Usuario

Una Historia de Usuario se puede descomponer de dos formas para el *testing*:

1. **Escenarios Dado-Cuando-Entonces:** Describen un comportamiento específico con condiciones previas (Dado), una acción (Cuando) y un resultado esperado (Entonces).

2. **Escenarios Panorámicos/Simples:** Pueden ser **oraciones sencillas y de alto nivel** que describen una funcionalidad general a probar (ej: "Enviar mail a un destinatario válido", "Eliminar mail en curso").

Ciclo de Vida del Diseño de Pruebas

Las pruebas se diseñan siguiendo un proceso estructurado:

Planificación -> Análisis -> Diseño -> Implementación -> Ejecución -> Reporte.

En la etapa de **Diseño**, se elaboran los **Casos de Prueba**.

Casos de Prueba

Son un **conjunto de inputs, condiciones y outputs** diseñados para verificar un objetivo o requisito particular. Pueden ser:

- **Positivos:** Verifican que el sistema funciona correctamente con entradas válidas.
- **Negativos:** Verifican que el sistema maneja correctamente entradas inválidas o situaciones inesperadas.

Frecuentemente se documentan en un formato tabular (cuadro).

ID	Título	Precondiciones	Datos de prueba	Pasos	Resultado esperado	Estado
001	Bloquear usuario luego de dos intentos fallidos	Que el usuario ya haya intentado loguearse dos veces fallidamente	Usuario: asdasd Contraseña: asdfdsa	1. acceder a la app 2. introducir los datos 3. presionar aceptar 4. verificar el comportamiento	El sistema debería bloquear el usuario y avisar al admin	pendiente

Implementación y Ejecución de Pruebas

En el **cuarto paso (Implementación)** y **quinto paso (Ejecución)** del ciclo de vida del diseño de pruebas, se lleva a cabo la creación de los scripts de prueba (si aplica) y la ejecución de los casos de prueba, recolectando todos los resultados obtenidos.

ID	Resultado obtenido	Estado	ID Issue	Comentarios
001	El sistema bloquea al usuario pero no envía la alerta	Falló	BUG001	Se probó en ambiente de test en versión 5.1